

Second Look

A two-minute test of how well a volunteer sees a creek, and a model that may only ask

Alex Velazquez and Rachel Selbrede, September 2026

Live: <https://second-look-79t.pages.dev> · Code: <https://github.com/alejandro-publius/second-look>

Abstract

Citizen observations of creeks are uneven. People notice smell, foam and colour and walk past concrete banks, dug-out channels, plants that do not belong and pipes. Second Look measures that, per person and per feature, with a two-minute test on 16 real photos, and stores the score with every observation the person later makes, in OneAquaHealth's own FHIR profiles. Vision models take the same test. A model may only ask a volunteer to look again, on a feature it passed, after the person has answered; code, never the model, writes the record. Four Claude models took the test three times each. Every model passed built banks, and no model passed plants that do not belong, where almost every answer was "can't tell": the plant photos show no water, and the instruction says to answer can't tell when no stream is in view. On 46 frames of open creek footage the gate dropped 29 of 64 candidate flags, each on a feature that model had not passed. The HL7 validator found 0 errors in 14 records checked against OneAquaHealth's guide. No person has taken the test yet, and this report makes no claim about people; a paid research panel may add sessions before the lock, and they would be reported once, after it.

1. The problem

People judge a creek the way they judge a park. Tidy and green reads as healthy. OneAquaHealth's project lead said it in the first workshop: volunteers catch smell, foam and colour, and walk past concrete banks, a channel that was dug out, and pretty plants that do not belong. So the best-looking creek can get the best rating and deserve the worst.

Professional surveyors fixed this long ago. In the UK's River Habitat Survey, "only surveys from accredited surveyors will be entered on the RHS database", and accreditation means attending a course and passing a test (RHS manual 2003, pages 3 and 20; see [docs/notes/sources.md](#)). Volunteers have never had that. Their observations arrive with no mark of how far to trust them.

Measure each volunteer, per feature, and store the measure with the data. The analyst sees "4 of 4 on built banks, tested Sep 23" beside an answer, never a blended grade or a probability. Follow-up questions are chosen by code from the answers, the person's scores and the weather, two at most: "It has not rained here for N days. Is anything coming out of that pipe?" The AI takes the same test as the people and earns the right to ask one question, feature by feature.

Every risk, what stops it, and the test that proves it | What can go wrong | What stops it, by construction | Proof | `---|---|---` | A volunteer walks past a built bank | The lesson teaches the four features people miss, and the test measures each one; the score is stored with every answer | `content/lessons/, core/scoring.py, core/tests/test_scoring.py` | Someone taps at random | Four items per feature, two present and two absent, so chance scores about 2 of 4; a low scorer who answers No is asked for a photo | `content/test_items.yaml, core/followups.py` rule `low_score` | A pretty creek gets the best rating | A good rating beside a reported built bank, sewage or plant that does not belong triggers one question asking the person to keep or change it | `content/followups.yaml` rule `rating_check, core/tests/test_followups.py` | A pipe report means nothing without the weather | The dry pipe question fires only after dry days from Open-Meteo, and is skipped when the weather is unknown | `core/rainfall.py, core/tests/test_rainfall.py` | The model invents a feature | Model output becomes a Flag through the gate or is dropped; a fuzz test throws arbitrary output at it | `core/gate.py, core/tests/test_gate.py` | The model was never good at that feature | A model may flag only a feature it passed on the same test as the people; the pass table is a committed file, and one from the fake client licenses nothing | `results/model_pass_table.json, core/tests/test_checker.py` | Duplicate and test pins fill the map | A precise pin within 30 metres of an existing spot is offered as that spot; names that read like a test are kept out of the counts and listed apart for a person to check; a coarse pin is never compared | `core/act.py, core/tests/test_act.py` | A record a city's systems cannot read | Sample records from both emitters, the Python API and the TypeScript Worker (14 files, walk records among them), are validated against their guide in CI, and golden vectors hold the live emitter to the same output | `scripts/fhir_validate.py, worker/test/golden.test.ts` | Someone edits history | A hash-chained audit log, checked by a script | `audit/, scripts/verify_audit.py` | Someone rewrites the audit log, last line included | Its last hash is stamped with OpenTimestamps once a day. A rewrite with fresh hashes still holds together, so the stamped line is compared with the log's own line; `/verify` does that too, and checks the whole chain again in your browser | `uv run python scripts/verify_audit.py`, then `uv run python scripts/ots_status.py`, which fails when the log no longer has a stamped line; `.venv/bin/ots verify proofs/audit-head-2026-09-24.ots` checks the stamp itself | The plan was written after the data came in | The `prereg-v1` tag object and `docs/analysis_plan.md` are stamped with OpenTimestamps, a public timestamp service that anchors hashes in Bitcoin; it is not our own chain. `/verify` shows each proof and its Bitcoin block once confirmed | `.venv/bin/ots verify proofs/prereg-v1.tag.ots, .venv/bin/ots verify -f docs/analysis_plan.md proofs/analysis_plan.md.ots` (`uv sync` puts `ots` in `.venv/bin`; it needs a Bitcoin node to finish), or `uv run python scripts/ots_status.py`, which checks the block against a public explorer; `proofs/README.md` | We fool ourselves with the statistics | The analysis plan is tagged before any data; every README number is checked against `results/` in CI; a synthetic result is cited only where the sentence says it is a simulation | `docs/analysis_plan.md` at `prereg-v1, scripts/verify_claims.py` | Judge mode leaks the answer key | Judge mode is shut until Sep 28 by a lock constant, and its answer route refuses before then on the Worker and in the Python API | `core/lock.py, apps/api/tests/test_study.py::test_demo_answer_is_shut_before_the_lock, worker/test/e2e.mjs` | A refresh loses

a session | The session resumes from the server; the creek check queues offline and sends later | [apps/web/Lib/offline.ts](#), [apps/web/tests/](#) |

2. Method

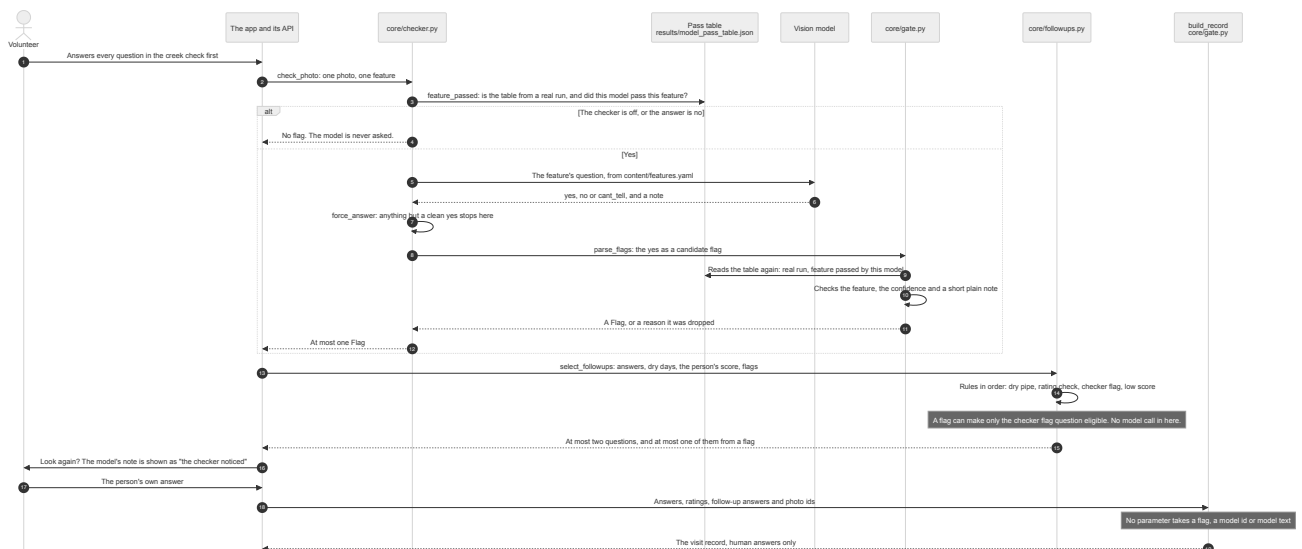


Figure: How a model's answer becomes a flag, or nothing.

2.1 The test

Sixteen photos, four per feature, two with the feature and two without, in a new random order for each person. The answers are Yes, No and Can't tell, and Can't tell counts as wrong. The score is kept per feature, out of 4, and stored in the record of the test sitting; a dated qualification for the test and a Provenance link carry it to every Observation the person makes. The question wording and the analysis were written into `docs/analysis_plan.md` and tagged `prereg-v1`, and the key was frozen as a hash in `results/key_hash.json`, on Sep 21, the day the test opened, before the first paid model run on Sep 23. The dates here are Pacific time.

2.2 The gate

A vision model can help a volunteer look again. It can never decide what is stored. Every model answer takes this path:

- 1. The person answers first.** The creek check asks the official app's questions in its order (`content/form.yaml`), and the answers are the person's own. Their wording is not yet checked against the app itself, so each question is marked draft wording.
- 2. The model is asked only where it passed.** `core/checker.py` reads the committed pass table, `results/model_pass_table.json`, and does not even ask about a feature that model did not pass on the same 16-photo test the volunteers take. A table that does not say `"real": true` licenses nothing.
- 3. Its answer is forced.** One photo, one feature, the frozen question wording. Anything that is not yes, no or can't tell with a short note becomes can't tell and counts as malformed (`force_answer` in `core/checker.py`).
- 4. The gate turns it into a flag or drops it.** `parse_flags` in `core/gate.py` keeps a `Flag` only for a known feature the model passed, with a sane confidence and a short plain note, and drops everything else with a reason in plain words. It never raises. On the footage run it dropped 29 of 64 candidate flags, each for a feature that model had not passed.
- 5. A flag can only make one question eligible.** `core/followups.py` picks the follow-up questions from the answers, the rain (`core/rainfall.py`), the person's scores and the flags, by the rules in `content/followups.yaml`: two questions at most, the model's at most one, and no model call inside it.
- 6. The person taps "I looked again" or "Skip", and no stored answer changes.** The question comes only for a feature the check asks about, so a flag on a dug-out channel asks nothing. In the creek check the same question has Yes, No and Can't tell, and that tap is kept beside the answers, never in place of one. The model's note is shown only as "the checker noticed" (`core/checker.py`, `apps/web/components/WalkFlow.tsx`).
- 7. The record is built from human inputs only.** `build_record` in `core/gate.py` has no parameter that could carry a flag, a model id or model text. `core/fhir_emit.py` writes the record under OneAquaHealth's profiles with the person's score attached, and `scripts/fhir_validate.py` checks it in CI.

Where it runs today: nowhere a person sees yet. `scripts/build_walks.py` sends the footage run's answers through the same gate when the video walks are built, and no flag on those clips passed it, so 0 of the 3 walks shows a checker question. The live creek check runs with the checker off (`CHECKER_ENABLED`), so both servers pass it no flags and no model is called. To see the gate act on the footage run, open `examples/footage-flag/`: one frame where it kept a model's flag, with the question that flag makes eligible, and one where it dropped a flag, with its reason in the gate's own words.

In the build measured here, 0 of the 3 walks carries a checker question: the gate kept no flag on the frames the walks use (`content/walks.yaml`). So today no model flag reaches a person, in the walks or on the live site.

2.3 What the AI cannot do

It cannot	Enforced by	Test
Decide anything stored	The record builder accepts human answers only	<code>core/tests/test_gate.py</code> , the fuzz test on stored answers
Speak on a feature it did not pass	<code>core/checker.py</code> refuses to ask, and <code>core/gate.py</code> drops the flag	<code>core/tests/test_checker.py::test_unpassed_feature_returns_nothing_even_when_the_model_is_confident</code>
Speak on a fake pass table	The gate reads "real" : true or licenses nothing	<code>core/tests/test_checker.py::test_synthetic_pass_table_never_licenses_a_flag_by_default</code>
Ask more than one question, or ask first	Follow-up selection is a pure function, two questions at most, the model's at most one, shown after the person answers	<code>core/tests/test_followups.py</code>
Put its own words in front of a person	Its note is shown only as "the checker noticed", cut to 160 characters	<code>core/checker.py</code> , <code>core/gate.py</code>
State a risk for a named site	Every health or ecology sentence comes from <code>content/approved_sentences.yaml</code> with a source	<code>core/tests/test_healthcard.py</code> , <code>core/tests/test_act.py</code>

3. Evidence

All numbers below come from files in `results/`, written by the scripts in `evals/` and checked against the text by `scripts/verify_claims.py` in CI. Failed runs stay in `results/` too.

3.1 The models on the test and on footage

	The 16-photo test people take	Frames from open creek footage
Pool	16 photos, 4 per feature, the frozen question wording	46 frames from 5 openly licensed videos in 3 countries, screened for people and text
Right answers, three runs of the 16 photos	Claude Haiku 4.5 31 of 48, Claude Sonnet 5 33 of 48, Claude Opus 5.5 33 of 48, Claude Fable 5.1 34 of 48; per feature, with intervals, in <code>results/benchmark_20260924T054939Z.json</code>	not measured: none of the frames has a label, so this column reports agreement
A checker that always answers No, the floor to beat	24 of 48	not asked
Right answers without the plant photos	Claude Haiku 4.5 31 of 36, Claude Sonnet 5 33 of 36, Claude Opus 5.5 33 of 36, Claude Fable 5.1 34 of 36	not asked
Which features each model passed	the table below	not asked: footage decides nothing, a pass is earned on the test
Agreement between models	not asked	Haiku 4.5 and Fable 5.1, the pair that agreed least, gave the same answer on 41 of 46 frames for built banks, 15 for a dug-out channel, 38 for invasive plants and 30 for pipes
Flags the gate stopped	not asked: an answer on the test is scored, never flagged	29 of 64 candidate flags dropped, 35 kept, because a model may flag only a feature it passed
One checker question about one frame, in cents	not asked	Claude Haiku 4.5 0.2, Claude Sonnet 5 0.4, Claude Opus 5.5 1.0, Claude Fable 5.1 2.3
Cost per 100 frames, each asked the four questions three times by all four models	not asked	51.3 USD, with direct calls at the full price

	The 16-photo test people take	Frames from open creek footage
Small bugs planted on purpose in four modules, the gate, the follow-up selector, scoring and the FHIR emitter, caught by core's tests (make mutation, mutmut); core/checker.py, core/rainfall.py and the Worker's TypeScript ports are not in this run	the gate caught 226 of 229, the follow-up picker 319 of 324, scoring 53 of 53, the FHIR record writer 1477 of 1543; each must catch 85.0 percent or more (results/mutation.json)	the same gate decides every footage flag

Which features each model passed on the 16-photo test: all four photos of a feature right in at least two of three runs (results/model_pass_table.json).

Model	Built bank	Dug-out channel	Invasive plant	Pipe running
Claude Haiku 4.5	passed	passed	did not pass	did not pass
Claude Sonnet 5	passed	passed	did not pass	did not pass
Claude Opus 5.5	passed	passed	did not pass	passed
Claude Fable 5.1	passed	did not pass	did not pass	passed

The full loop, from a desk: 3 video walks from 3 countries, each ending in a FHIR record made on the phone. The HL7 validator checked 14 records against OneAquaHealth's guide, 2 of them walk records, with 0 errors.

The test runs as a pre-registered study that stays open: it is the volunteer's own calibration step and the judges' first door, and a paid research panel may add sessions before the lock. No session from a person has arrived yet, so there is no human row here; sessions that arrive are reported once, after the lock, whatever they show.

3.2 The benchmark and its size

The same 16 photos, 4 per feature (2 with the feature, 2 without), run 3 times more by evals/benchmark.py, a second set of calls from the ones that wrote the pass table. Each cell is right answers of all answers, with the Wilson 95 percent interval in percent. Can't tell counts as wrong. The file is results/benchmark_20260924T054939Z.json.

Model	All 16 photos	Built bank	Dug-out channel	Invasive plant	Pipe running	Can't tell
Claude Haiku 4.5	31 of 48 (50 to 77)	12 of 12 (76 to 100)	10 of 12 (55 to 95)	0 of 12 (0 to 24)	9 of 12 (47 to 91)	35 percent
Claude Sonnet 5	33 of 48 (55 to 80)	12 of 12 (76 to 100)	12 of 12 (76 to 100)	0 of 12 (0 to 24)	9 of 12 (47 to 91)	27 percent
Claude Opus 5.5	33 of 48 (55 to 80)	12 of 12 (76 to 100)	9 of 12 (47 to 91)	0 of 12 (0 to 24)	12 of 12 (76 to 100)	31 percent
Claude Fable 5.1	34 of 48 (57 to 82)	12 of 12 (76 to 100)	10 of 12 (55 to 95)	0 of 12 (0 to 24)	12 of 12 (76 to 100)	27 percent

Twelve answers per feature is small. An interval such as 55 to 95 percent means the test cannot tell a good model from a very good one. Read the intervals, not the point numbers.

On creek footage. The four models also answered every feature question on 46 frames from 5 openly licensed videos in 3 countries. No frame has a label (0 labelled), so this measures agreement between models, not accuracy. The pair that agreed least, Haiku 4.5 and Fable 5.1, gave the same answer on 15 of 46 frames for a dug-out channel. Four frames drawn by code, a white one, a black one, a room made of flat rectangles and a screenshot of text (evals/fixtures/__init__.py), got can't tell as each model's majority answer over three runs, on every feature (results/footage_latest.json, under adversarial; the run kept the majority answers, not each reply).

3.3 How the models fail

- **A run once measured our config instead of the models.** The first paid sweep sent an answer tool whose allowed values came from YAML, where an unquoted yes and no are booleans. The models were told to answer true or false, and 73 of 144 answers were malformed. The values are now quoted and pinned by evals/tests/test_model_sweep.py::test_the_answer_tool_allows_exactly_the_three_words_as_strings; the run done again had 0 malformed. Both runs stay in results/.
- **No model passes invasive plants.** In the sweep behind the pass table, 47 of the 48 answers on the plant photos were can't tell. The plant photos are close views of a plant with no water in them, and the instruction says to answer can't tell when the photo does not show a stream. The notes show it: several name the plant, for example "possibly invasive Himalayan blackberry, but no stream is visible", and still answer can't tell. So this result measures our photos and our instruction as much as the models. The photos and the question wording are frozen with the tagged analysis plan, so changing them would be a deviation. The instruction is not in the plan: it is ours, in evals/models.yaml, and changing it would need a new paid run and a new pass table.
- **Can't tell is common.** Answers per feature in that sweep, all four models together:

Feature	Answers	Yes	No	Can't tell	Right
Built bank	48	24	24	0	48
Dug-out channel	48	20	24	4	42
Invasive plant	48	1	0	47	1
Pipe running	48	22	23	3	43

to a volunteer's pipe, marked EXAMPLE | [core/fhir_referral.py](#) | | Not theirs: iNaturalist's public API, under its [terms](#); each observation keeps its observer's licence | One context line per creek, research grade sightings of the region's listed invasive plants near its spots, shown once a finished check on that creek has answered the plant question, never in the check and never counted; the Bay Area list waits on a check, so it reports none yet. | [docs/adr/0011-inaturalist-context.md](#) |

One creek visit becomes these resources ([docs/fhir_mapping.md](#)):

Resource	Profile	What it holds
Location, three nested	their Location0ah	creek, then reach (partOf creek), then spot (partOf reach, with position). Each has an identifier, a name and mode = instance, as the profile requires.
Practitioner	base R4	the volunteer, identified only by a random contributor token. One qualification coded second-look-test, period from the test date to 90 days later, issuer our Organization.
QuestionnaireResponse, test sitting	base R4	one item per feature with the score out of 4, computed by code. Authored by the Practitioner.
QuestionnaireResponse, visit	base R4	one item per answered check item. Link ids are the stable ids from content/form.yaml.
Observation, one per answered item	their ObservationIndicators0ah	status final, subject the spot, performer the Practitioner, effectiveDateTime, derivedFrom the visit response, a coded or quantity value.
Provenance	base R4	target every Observation of the visit, agent author = Practitioner and assembler = our software Device, entity source = the visit response and the test sitting. This is the path from any answer to the score of the person who gave it.
Organization, Device	base R4	us, and the software.
Bundle, type collection	base R4	everything above. The emitter will also produce a transaction Bundle with conditional creates for the sandbox mirror.

- `Observation.value`: their `Temporary0ahSystem#present` and `#absent`. "Can't tell" is our `second-look#cant-tell` because their system has no general unknown code.
- `Observation.category`: their indicator groups. Built banks and dug-out channel sit under `#morphology` (their spelling), invasive plants under `#invasiveOrganisms`, pipes and water height under `#hydrology`. The profile does not bind category, so this is additive.
- `Observation.code`: our feature codes `artificial-bank`, `dug-out-channel`, `invasive-plant`, `pipe-running`. Their profile binds code to `0ahIndicatorsNoHealth0ahVs` with strength preferred, so a code outside it produces an information note, not an error. The validator confirmed this. Water height uses their own `#hydrology` code directly.
- Coding displays must be the code system's own display. The words a person sees on screen live in `content/form.yaml`, never in the coding.
- Units are UCUM (m for water height).

5. Limitations

- One labeller. The gold labels came from the picks file Alex wrote with the planner, a Claude chat (commit 81e62ed), and no second, blind label exists yet, so read every accuracy as agreement with this key ([docs/DATA_CARD.md](#), [docs/deviations.md](#)).
- Four photos per feature is coarse: it shows a person what to practise and flags an answer worth a second look, but it is too coarse to weight votes by feature. In our simulation with made-up people ([results/consensus_coarseness.json](#)), weights from each feature's own photos did worse than a plain majority in all 5 skill patterns.
- The official app has no question for a dug-out channel, so the creek check asks none: that score is kept but stands beside no answer, and the city measure it would lead to waits for one. Plants that do not belong have no city measure of their own.
- The photos come from open collections in several countries and seasons, not from the creeks a Berkeley visitor will stand in.
- The footage labels come from the videos' own descriptions, and almost no openly licensed description names a feature, so the footage result leans on agreement between models, which is not accuracy.
- The citizen observer is modelled as a FHIR Practitioner, for want of a better fit in the guide; our proposal says so.
- Judge mode is shut until Sep 28, so the answer key cannot leak before then.
- Their sandbox's name, `sandbox.hl7europe.eu`, stopped resolving on Sep 23. /two can show their record only from a copy the Mac fetches while it answers, so for now it shows ours alone, and the re-push to their sandbox waits until it answers again.
- The tagged plan did not plan recruitment, and nobody has taken the test yet. A paid research panel may add sessions before the lock if Alex launches it (a logged deviation); without it, whoever opens the link is whoever opens the link. The panel study has had no ethics review: it is an anonymous usability test that keeps no name or contact, and whether a paid one needs a review is checked before it launches.
- The AI numbers come from paid calls on Sep 23, Pacific time: four models, three runs each, on 16 photos, once for the pass table and once more for the right-answer counts, plus the footage run. That is small. Read the intervals in the benchmark file, not the point numbers.
- English only. A Spanish draft exists and stays out of the build until a fluent person signs it.

The labels have a limit of their own. The test key came from the photo picks, with one labeller, and no blind second label exists yet ([docs/DATA_CARD.md](#)). Until one does, read every accuracy figure here as agreement with this key.

6. Contributions

1. **A measured observer, carried in the record.** A two-minute test gives each volunteer a score per feature, and OneAquaHealth's own profiles carry it, through a qualification and Provenance, to every Observation that person makes. A city analyst reads each answer beside how well its observer sees.
2. **A model that may ask and never decide.** A model is licensed feature by feature by the same test people take, from a committed pass table. Its output becomes a flag through one gate or is dropped, and the record is built from human inputs only.
3. **Measurement that shows its failures.** The pass table, the benchmark with its intervals, the run that measured our own config, and the plant photos that no model could judge are all in `results/`, with the cost of every call.
4. **Open work, and work given back.** The code is MIT. What we contributed back, as the README lists it:

Sent to OneAquaHealth's implementation guide on 2026-09-24, in the open:

- [hl7-eu/oah pull request 5](#): our citizen observer example, five FSH files that build inside their guide with SUSHI 3.20.1 at no errors, and the proposal as a page. It asks which resource should stand for a citizen observer.
- The proposal in that pull request, for carrying observer quality in their guide, with three gaps our validator runs found: no profile for the person or the trail from an answer to them; a volunteer modelled as a Practitioner for want of a better fit; `SpecimenOah.collection.collector` allowing only a `PractitionerRole`. Its FSH builds inside their guide in CI. [docs/ig_proposal.md](#).
- [hl7-eu/oah issue 6](#): their temporary code system spells one code `morphology`. We kept their spelling so our records validate.
- [hl7-eu/oah issue 7](#): `SpecimenOah.collection.collector` allows only a `PractitionerRole`, which leaves out a laboratory and a volunteer who takes a sample.
- [hl7-eu/oah issue 8](#): their sandbox's name stopped resolving on 2026-09-23, with the evidence from their own nameserver.
- A read only MCP server over our own records, so any software agent can ask for a creek's records with the resource ids behind every answer. [examples/mcp/README.md](#).

7. How to check this report

- `make judge-check` runs the tests with no key and no network.
- `make verify-claims` checks every number in the README and the docs against `results/`.
- `make report-pdf` builds this file again from its sources; a test fails when the PDF is older than the README sections and results it was built from.
- The model card, the data card and the threat model are `docs/MODEL_CARD.md`, `docs/DATA_CARD.md` and `docs/THREAT_MODEL.md`.

AI coding tools wrote most of the code and the text, from written briefs, and every change was checked by `make check` before it was committed. The people set the direction and made every decision that needs a person, as the README says under "How this was built".